

4 Signal Search - Audio matching

If we don't have an exact recording of audio that's in the library, we can't match exactly the fingerprints that we would have heard. What happens when I want to identify live music, which isn't perfectly synchronized with the recording in the database?

Instead of extracting fingerprints, we can extract features. Then we make a database (like before), and do a matching algorithm. We'll be using Dynamic Time Warping!

Extract Features: chroma

We want to reduce dimensionality of our data, while capturing relevant information. This depends on the application - maybe volume is relevant or not, same with pitch and timbre.

We usually record data in the form of which music "note", which doesn't depend on the octave. I.e, we sort notes into: AA#BCC#DD#EFF#GG#

To do this, we identify the fundamental frequency. There is an envelope that determines the shape of the harmonics, which is the timbre.

We take the STFT ($\frac{p}{2} + 1$ frequencies) and label bins for each chroma (12 or) music notes, depending on audio source and application , then collapse and make a matrix which is 12 (# of pitches) by m (number of audio frames).

To collapse to the chroma, we take the log frequency by integrating the power over each pitch band (range of frequencies for notes on piano).

If we take A4 as our reference,

$$(\text{pitch}) = 0 \cdot 2^{\frac{p-6}{12}}$$

We delineate bins by $(\text{pitch} \pm 0.5)$, since boundaries are equidistant on a log scale, not linear.

We can make volume invariant by (after summing all the bins to get chroma) normalizing to have Euclidian length 1 of the 12 dimensional vector.

The chroma representation is useful, because we can change key by cyclic shift - stepping up and down by a half step.

We have two problems that might arise:

We are storing the power spectrum for 12 chromas at each frame, but we don't know which frame we're on.

We are also comparing with real live recordings, so there could be dynamic tempo changes.

Finding the unknown offset(the start)

Lets start with one problem: finding where to start if the tempo is the same.

We can calculate a cost matrix by taking the inner product between the search matrix and the reference matrix (both are normalized). We then make a cost matrix

$$C(i, j) = 1 - \frac{j}{\text{cosine similarity}}$$

The lower an element, the closer they are to each other. A diagonal line of these shows that the songs align, and the start of that line is the offset.

Finding similarity with different tempos

Lets find the minimum path from bottom left to top right, only moving up or right.

5	1	3	9	1
2	8	9	4	6
9	3	5	6	4
4	3	1	7	3

Imagine that going across is different frames in music sequence 1, and going up is different frames in music sequence 2. We are taking the dot product between the chroma features of each sequence for that pair of frames.

The minimum path length gets 29.

The type of transitions allowed is important.

Dynamic Time Warping

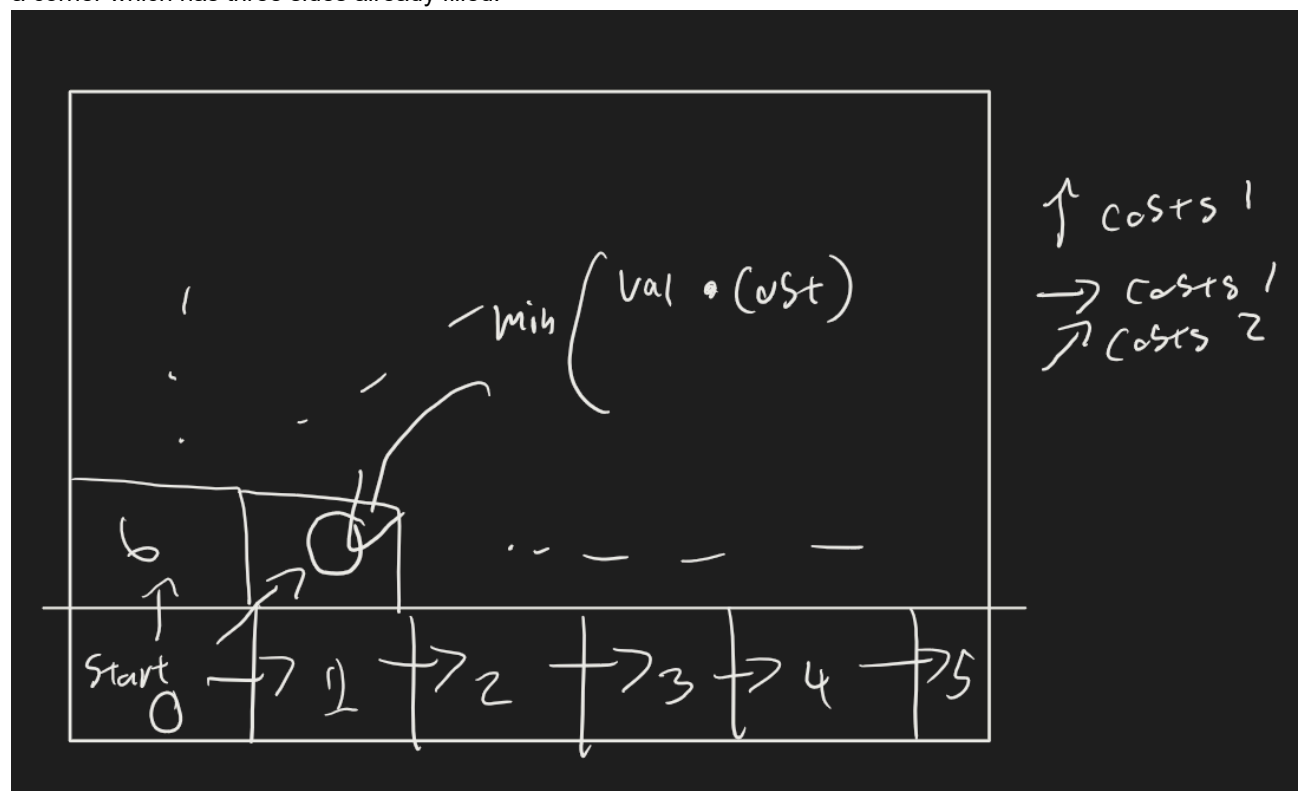
Given a cost matrix and a set of transitions (and maybe some weighting or cost for each type of movement), find the minimum path.

We take in a cost matrix as an input. We then calculate a cumulative cost matrix, which is the minimum cost to get to that point in the path. To get to the next point, we find the minimum cost from any of the adjacent squares. We then propagate that through the matrix.

Each step we get

$$\begin{aligned} & i-1, j + C_{i,j}^1 \\ & i, j-1 + C_{i,j}^2 \\ & i-1, j-1 + C_{i,j}^3 \end{aligned}$$

If we go from left to right starting from the bottom, the bottom row is easy and just steps going right. The rows are easy because it's just steps going up. When we go left to right then increase 1 row, we are always calculating either an edge, which is easy, or a corner which has three sides already filled.



At the same time, we calculate a back trace matrix. This just stores the ID for each transition, the last step to end up in the position i, j . I.e. maybe up would be 0, left would be 1, diagonal would be 2.

We know the optimal path score in the top right, and we can back trace from the back trace matrix how to get that score.

The minimum path finds where the signals align, starting from the same spot. This curvy path is the time differences.

Combining them together

We have a new thing called the subsequence dynamic time warping.

We pass in the cost matrix between the query features and reference features

We define the allowable transitions, and the transition weights

The query reference is much shorter than the reference.

We return the lowest cost subsequence path (starts in the middle, ends in the middle).

There are 4 steps.

1) Calculate the cumulative cost matrix D

The i,j th element is the cumulative cost of the optimal path from $(0,?)$ to (i,j) . The $?$ is the start of our query, which can be any frame offset in the reference.

The difference from the DTW version: we initialize it differently: The entire first row is initialized to match c .

$0, m = C0, m$, where $m = 0 \dots 1$. We don't penalize a path for starting in the middle.

2) Identify the endpoint

We can end anywhere, not just the very end. We look at the top row, and see which has the lowest cumulative path cost. The column with the minimum element has the lowest total path score, so that is where we end. The index of the minimum value in the cumulative cost matrix is the end point of the subsequence path

3) If we need the alignment, construct the backtrace matrix at the same time

Just do it.

4) If you need the path, back trace through b.

The starting element (column number in the first row) is the offset between paths. That tells you how far into the song the query starts.